Xi'an Jiaotong-Liverpool University
西交利物浦大學

School of Advanced Technology

ISA104 AI Application Design

Lab 3: Building a RAG and Agentic AI pipeline with XIPU AI

In this lab, you will use XIPU AI and RAG to design and prototype an agentic AI application. You will work with the platform's agent and knowledge-base functions and focus on agent design, knowledge grounding, capabilities, multi-agent organisation, and evaluation.

Central idea
Do not add RAG, tools, MCP, skills, or multiple agents simply because they exist. Start with an applicable scenario and add only the components that improve the agent's ability to complete that task reliably.

What you should have at the end of the lab

- Familiarity with the XJTLU XIPU AI platform.
- A first working XIPU AI agent.
- A knowledge base for RAG linked to the agent, the skill-creator skill attached and tested, and one MCP capability connected and tested.
- A small multi-agent architecture.
- A completed agent-loop trace showing how a new observation changes the next action or plan.
- An evaluation table containing relevant test cases.

Access

Log in using your XJTLU account: Open XIPU AI

Further XIPU AI guidance

XJTLU Knowledge Base → AI Features → XIPU AI Platform:

knowledgebase.xjtlu.edu.cn/category/ai-features/xipu-ai-platform/93/

Task 1: Explore XIPU AI and the Agent Center *(~10 min)*

Before building anything, inspect how existing applications differ. Open XIPU AI and locate the Agent area/Agent Center.

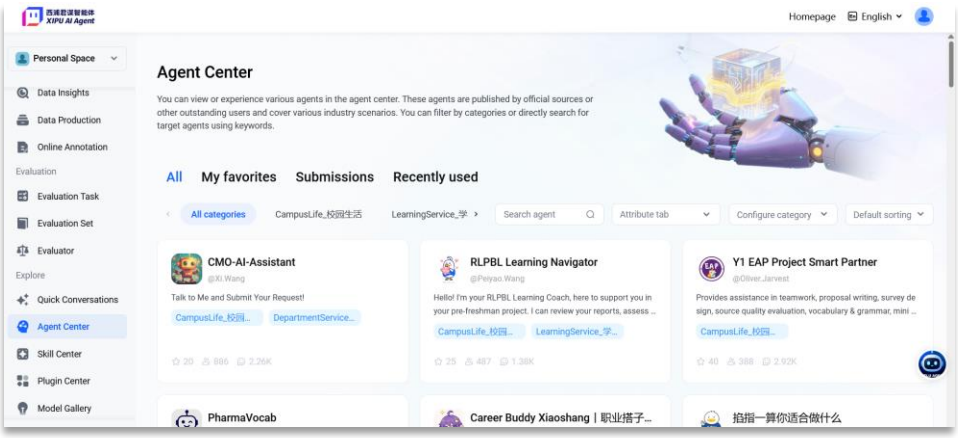


Figure 1. Agent Center on XIPU AI platform.

Find two contrasting examples:

1. A Q&A or knowledge-focused agent that primarily answers questions.
2. An image-creation or other action-oriented agent that uses a generation capability/tool.

For each example, record what you observe:

Example	User input	Knowledge/context	Tool/action	Output	Likely failure mode
Q&A / knowledge agent					
Image-creation agent					

Think before you build
A Q&A agent may need a strong knowledge base but no external action. An image agent may need very little domain knowledge but does need a generation tool and a clear transformation workflow. “Agentic” does not mean “use every feature”, but should obey precise definitions for clear outcomes and accountability.

Task 2: Design and build your first agent (~20 min)

In this lab, we will implement an agent which will support students participating in the fictional Smart Campus Innovation Sprint exercise.

Agent name	ISA104_lab3
Primary user	ISA104 student teams using the fictional Smart Campus Innovation Sprint exercise.
Applicable scenario	Help a team understand sprint rules, plan required deliverables, check compliance, and create an approved visual brief when requested.
Typical input	Questions about deadlines/rules; a project idea or status; draft deliverable text; or a request for a poster/image.
Desired output/action	Grounded answers, compliance checklists, a next-action plan, or a structured visual brief

	followed by image generation when explicitly requested.
Knowledge required	Smart Campus Innovation Sprint 2026 rules plus the CampusFlow practice project profile or another short project brief.
Capabilities required	RAG for sprint facts; skill-creator for the Skill exercise; image generation only if a visual capability is available and useful. The Time MCP is deliberately introduced later, inside the multi-agent exercise.
Boundaries	Do not invent missing rules, do not treat the historical 2025 deadline as current, do not upload personal/sensitive student data, and do not invoke image generation unless a visual is requested.
Success criterion	Pass the fixed evaluation set, reject unsupported questions, replan when constraints change, and select the knowledge base, skill, or tool only when required. In Task 6, also route a mixed RAG + time-conversion request correctly between two agents.

Architecture sketch.

Build the first agent and write its prompt

Create a new agent in XIPU AI with the following **prompt**.

```
# Role
You are the Smart Campus Sprint Coach for ISA104 student teams. You help teams understand the Smart Campus Innovation Sprint 2026, plan compliant work, and select the appropriate XIPU AI capability when needed.
## Goal
- Help the user produce a compliant next-action plan or grounded answer using the sprint rules.
- Maintain the user's goal when new observations appear and revise the plan when a hard constraint is no longer satisfied.
## Skills and process description
1. Use the connected sprint knowledge base whenever the request depends on sprint facts, dates, eligibility, judging, deliverables, locations, policies, or the CampusFlow practice-project status.
2. Apply the compliance-check procedure when a plan or draft must be compared with sprint requirements. Use image generation only when the user explicitly requests a visual.
3. For a broad goal, first create a short plan of 3-5 steps and identify the next useful action.
4. Execute or request the next observable action, inspect the resulting information, check it against the goal and constraints, and then continue, replan, or stop.
5. If a new observation invalidates the current plan, revise the plan before proceeding. If required information is missing or a capability fails, state the limitation and choose the safest useful fallback.
## Output format
Use concise headings and bullet points. For compliance checks, return a table with: Requirement | Evidence | Status | Action needed. For planning, show the current plan, the latest observation, and the next action. For visual requests, show the verified factual brief before invoking image generation.
## Constraints
- Treat the current 2026 sprint information in the connected knowledge base as authoritative; do not replace it with the historical 2025 note.
- Do not invent passwords, contacts, rules, approvals, or facts that are absent from the knowledge base.
- Do not invoke image generation or another tool when a direct grounded answer from the knowledge base is sufficient.
- Do not advise uploading personal or identifiable student data to an external AI service.
- Keep consequential or irreversible actions subject to explicit user approval.
```

Test the agent with three simple prompts before adding any knowledge or tools. Record one thing it already does well and one limitation that you expect the next tasks to solve.

Test prompt	Your observation
What is the maximum team size?	
Help us plan a concept note for an AI system that reduces food waste.	
Create a poster image for the project.	

Task 3: Create a knowledge base for RAG (~20 min)

Download and upload the accompanying file:

Sample Knowledge Base - Smart Campus Innovation Sprint.docx.

It is fictional and is designed specifically for testing retrieval from text, a table, rules, and an outdated historical note.

In XIPU AI:

1. Open *Knowledge* and create a new knowledge base.

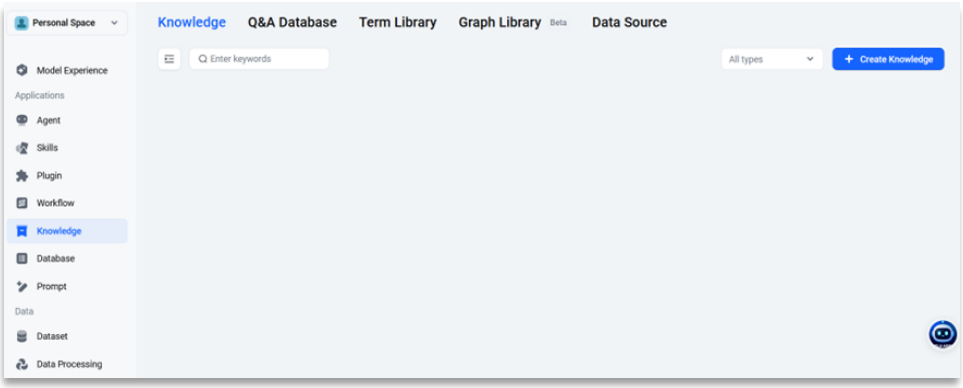


Figure 2. Open the Knowledge Base area in XIPU AI.

2. Upload the sample document (*Import file*) and wait until processing is complete.

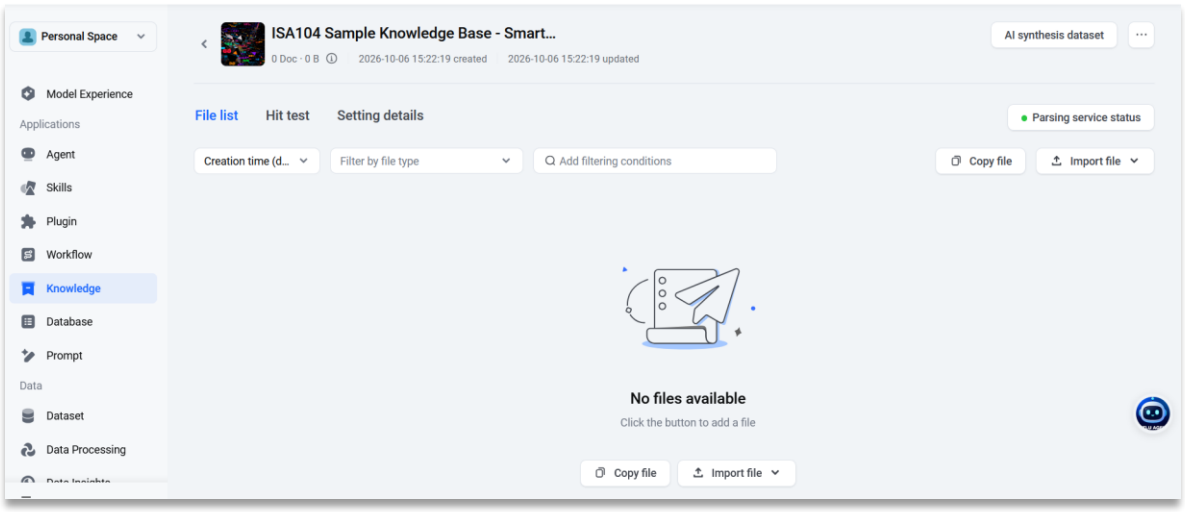


Figure 3. Manage a knowledge base and upload the sample document.

3. Use *Auto chunking* for reliable results. What influence in *Custom Chunking* would you expect for very small *Maximum chunk length*?

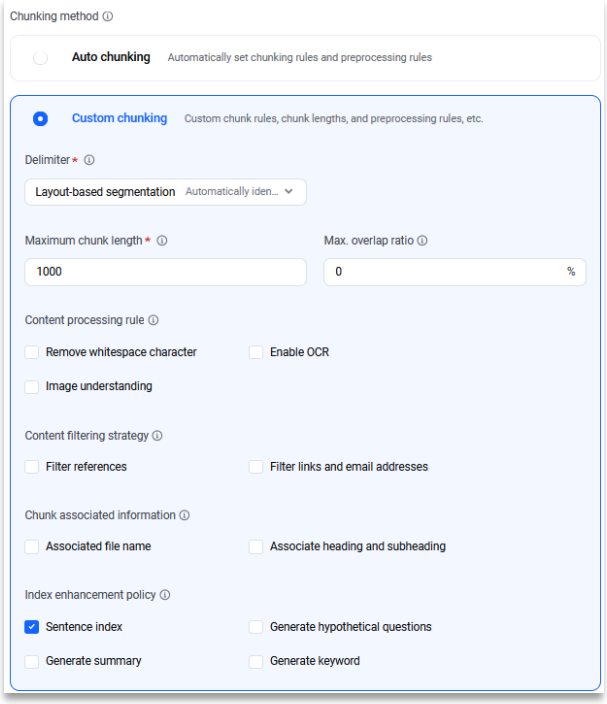


Figure 4. The platform lets you choose between Auto and Custom Chunking.

4. Use the knowledge-base *Hit test* function to ask questions before connecting it to your agent.

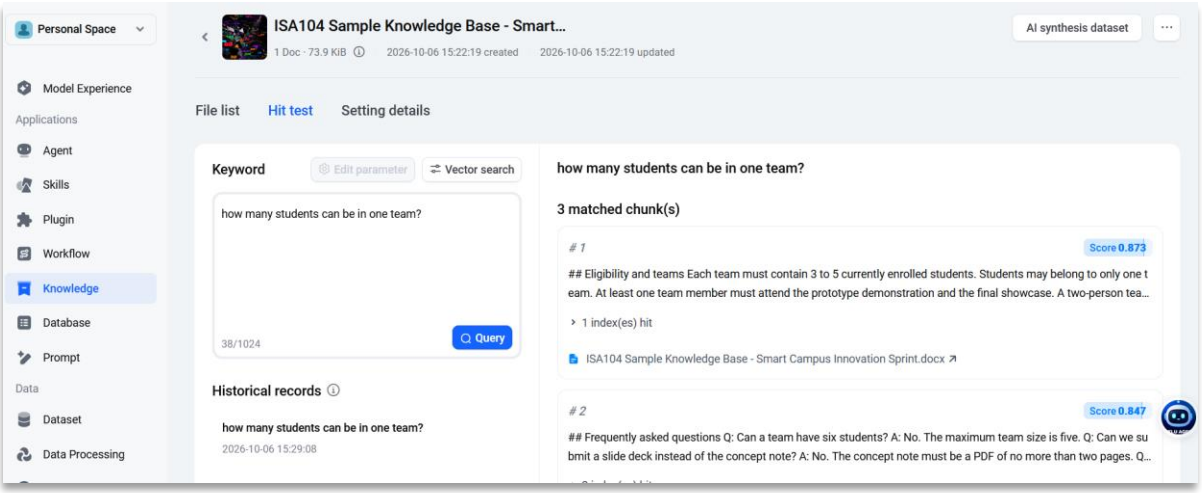


Figure 5. Use the Trial function to test the knowledge base before connecting it to an agent.

5. Connect the knowledge base to your agent and repeat the tests through the agent.

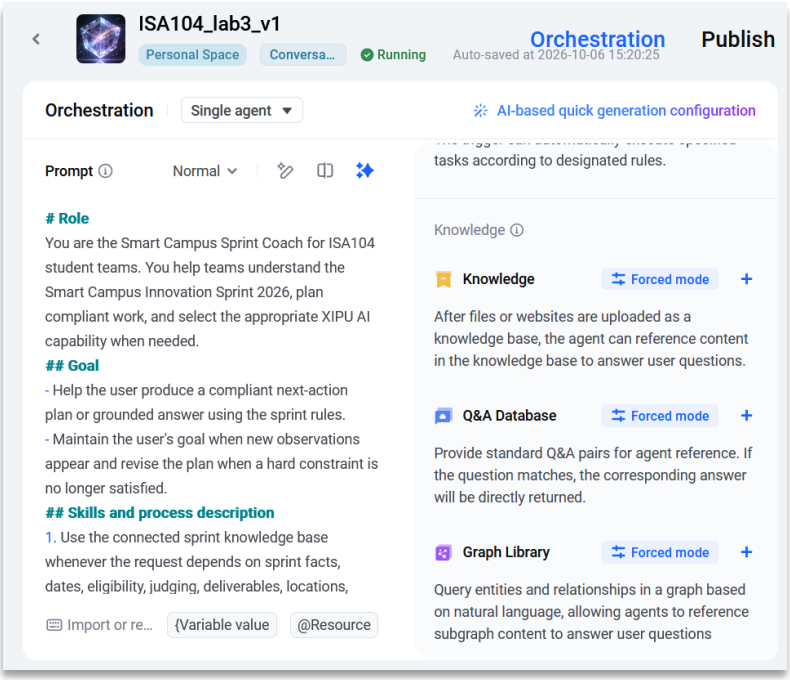


Figure 6. Expand Knowledge Base Configuration in the agent and select the knowledge base.

Test the RAG system on the knowledgebase with respect to the following aspects:

Test type	Question	Your result / observation
Direct fact	How many students may be in one team?	
Table extraction	What percentage of the judging score is assigned to technical implementation?	
Negative rejection		
Counterfactual robustness		

Test type	Question	Your result / observation
Noise / outdated information		

Task 4: Trace an agent loop and test autonomy (~10 min)

An agentic system should do more than answer one question. It should pursue a goal, choose a next action, observe the result, and use that observation to continue, verify, or revise its plan. In this task, trace that loop using your XIPU AI agent and the sample knowledge base.

Use the practice project profile in the sample knowledge base. Give the agent this goal: "Prepare a compliant next-action plan for the CampusFlow project for the Smart Campus Innovation Sprint."

1. Ask the agent to produce a short plan of 3-5 steps before it starts solving the goal.

2. Introduce a new observation: "One team member has left. CampusFlow now has only two students." Ask the agent to continue pursuing the same goal.

3. Check whether the agent changes its priorities rather than blindly continuing the original plan.

Answer briefly:

• What changed in the plan after the new observation?

• Which part of the interaction demonstrates agentic behaviour rather than one-shot generation?

Task 5: Use Skill Creator to create a reusable skill (~15 min)

XIPU AI separates reusable Skills from callable tools and MCP connections. In this task, focus on the easiest Skill workflow: attach skill-creator and use it to draft and test one small reusable skill.

Component	Meaning in this lab	Example
Skill	A reusable procedure or specialised instruction that can be applied repeatedly.	Use skill-creator to create a Submission Readiness Checker that follows the same compliance-check procedure each time.
Tool	A callable capability that produces information or performs an action.	Generate an image, calculate, search, or convert a time after a plugin/MCP connection exposes the operation.
MCP	A standard connection through which an agent can access external tools/resources exposed by an MCP server.	In Task 6, the Time MCP gives a dedicated utility agent access to time and time-zone conversion.

Part A - Add an existing skill from Skill Center

1. Open your agent in XIPU AI and select *Add skill*.

2. Stay on the Skill center tab, find *skill-creator*, and add it to the agent. In the screenshot below, "*Remove*" indicates that skill-creator is already attached.

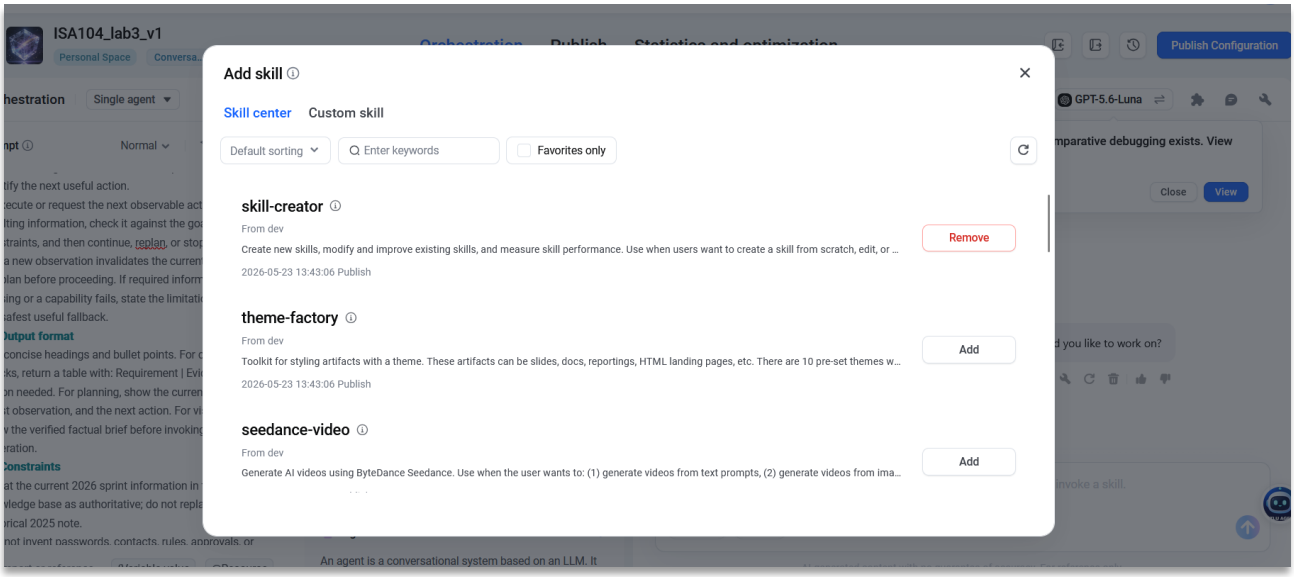


Figure 7. Add skill -> Skill Center. The skill-creator skill can be attached directly to the agent.

Part B - Use Skill Creator to design a reusable domain skill

Ask the agent to create a small skill named Submission Readiness Checker. A suitable request is:

> Create a reusable skill called submission-readiness-checker.
> Purpose: assess whether a student project is ready/eligible/compliant with the requirements available to the agent.
> Trigger it when the user asks whether a project is ready, eligible, compliant, complete, or what is still missing.
> Process: use the applicable knowledge source; list the relevant requirements; compare each requirement with the evidence provided; mark each item SATISFIED, MISSING, CONFLICT, or UNKNOWN; then recommend the next action.
> Output: overall status READY, NOT READY, or NEEDS INFORMATION, followed by Satisfied, Problems, Missing information, and Recommended next action.
> Constraints: do not invent missing rules or project evidence; treat unknown evidence as unknown; prefer current authoritative information over outdated notes.
> Also propose three realistic test prompts for the skill.

Test the proposed skill before considering it finished:

Test prompt	Your result / expected behaviour
CampusFlow now has only two students. Is the project eligible to continue?	
We have finished our project. Are we ready for submission?	
CampusFlow still has 3 students, but its concept note has no evaluation measure. Is the concept note ready?	

Task 6: Build a multi-agent system with a Time MCP specialist (~25 min)

A single agent can access many resources, but giving every capability to one agent makes routing and debugging harder. In this task, switch XIPU AI from Single agent to Multi-agent orchestration. Use the main node as a Coordinator for Smart Campus Sprint questions and create a second Time agent that is called only when a current-time or time-zone operation is required. Each node has an Applicable scenario for routing, an Agent prompt for behaviour, and its own Knowledge and Skills / Plugins.

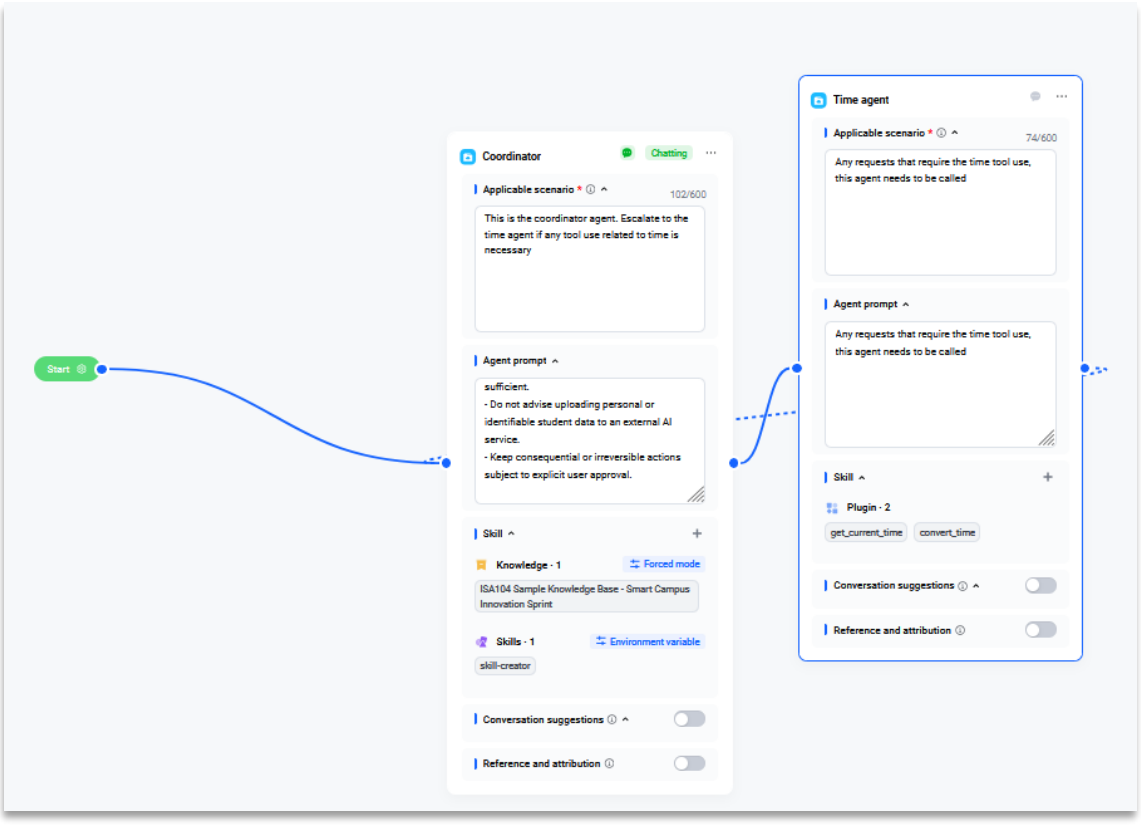


Figure 8. XIPU AI Multi-agent orchestration used in this lab. Start enters the Coordinator, which has the Smart Campus knowledge base and skill-creator. The Time agent is a specialist node with the Time MCP tools. The Applicable scenario field describes when routing to each agent is appropriate.

Target architecture: Start → Coordinator → Time agent (only when a time operation is required)

Agent	Main responsibility	Resources attached	Applicable scenario
Coordinator	Default agent. Understand the request, retrieve/verify sprint facts, plan or check compliance, and delegate a time operation only when necessary.	Smart Campus knowledge base; skill-creator remains attached from Task 5.	Use for Smart Campus Sprint rules, deadlines, eligibility, deliverables, judging, CampusFlow status, planning, compliance, and mixed requests that first require sprint facts.
Time agent	Specialist for current-time and time-zone operations. It does not decide sprint facts.	Time MCP plugin: get_current_time and convert_time. No sprint knowledge base.	Use only when the request requires the current time or a time/time-zone conversion. Receive a verified source time from the Coordinator when the value originates from the sprint knowledge base.

Step 1 - Switch to Multi-agent mode

In the Orchestration drop-down, select Multi-agent. We start with two specialist agents, created by *Add Node*.

Step 2 - Configure Agent 1: Coordinator

Applicable scenario:

Default entry agent for ISA104 Smart Campus Innovation Sprint requests. Use for sprint rules, deadlines, eligibility, deliverables, judging, CampusFlow status, planning, and compliance. If the request also requires current-time information or a time-zone conversion, first verify the relevant date/time from the knowledge base, then redirect to the Time agent.

Agent prompt:

```
# Role
You are the Coordinator for the ISA104 Smart Campus Sprint assistant. You are the default agent for Smart Campus Innovation Sprint requests.

## Goal
- Resolve the user's sprint-related request using verified information.
- Delegate a time operation to the Time agent only when that specialist capability is actually required.

## Skills and process description
1. Identify the user's goal and decide whether the request is factual Q&A, planning, compliance checking, skill creation, or a mixed task.
2. Use the connected Smart Campus knowledge base whenever the request depends on sprint facts, dates, eligibility, judging, deliverables, or CampusFlow status.
3. For broad goals, create a short plan, execute the next observable step, check the result against the goal and constraints, and replan when new information invalidates the current plan.
4. Use skill-creator only when the user asks to create, modify, improve, or evaluate a reusable skill. Do not invoke skill-creator for ordinary sprint Q&A.
5. If the request requires current-time information or a time-zone conversion, first verify any sprint-derived date/time. Then redirect to the Time agent and pass the exact source date, clock time, source time zone, and requested target location/time zone.
6. If no specialist capability is needed, answer directly from the verified information.

## Output format
Use concise headings and bullet points. For factual answers, state the verified answer directly. For planning, show the current plan, latest observation, and next action. Before redirecting to the Time agent, preserve the verified source time exactly.

## Constraints
- Treat the current 2026 sprint information in the connected knowledge base as authoritative.
- Do not invent rules, dates, contacts, passwords, approvals, or missing facts.
- Do not perform time-zone conversion yourself when the Time agent is available.
- Do not invoke skill-creator unless the user is actually asking to create or improve a skill.
- Keep consequential or irreversible actions subject to explicit user approval.
```

Step 3 - Configure Agent 2: Time agent

Add a tool under the *Skill*-section of the Time Agent. Enable the MCP only filter, and attach the Time MCP plugin to the Time agent. The Time MCP exposes get_current_time and convert_time. Do not attach the sprint knowledge base to this specialist: the Coordinator should verify sprint facts first and pass only the verified time information that needs conversion.

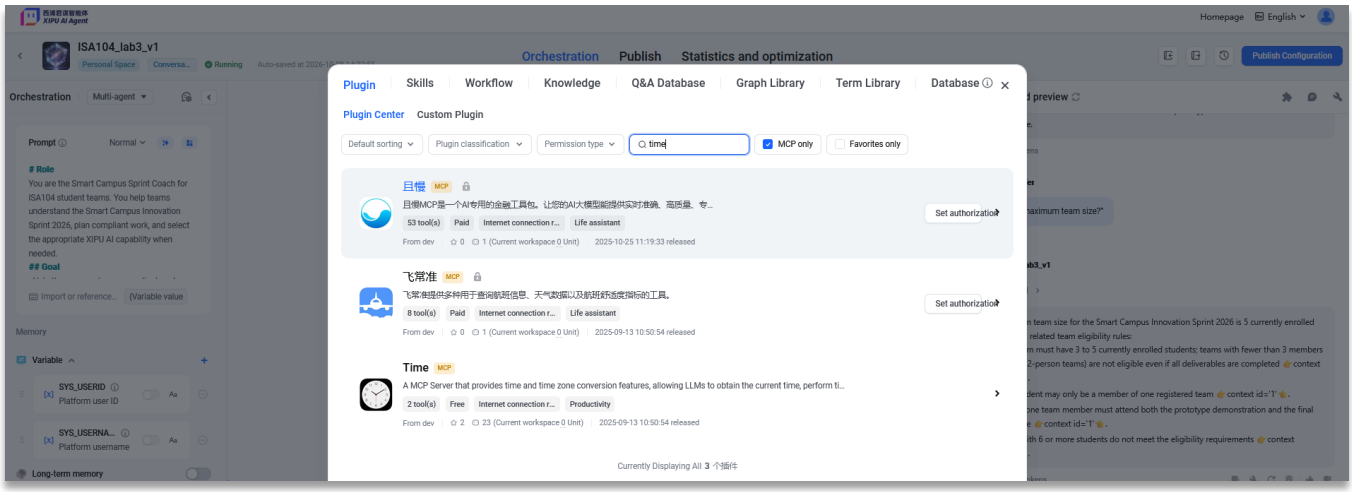


Figure 9. Skill view of Time agent. The Time plugin is marked MCP and exposes current-time and time-zone operations for the Time agent.

Applicable scenario:

```
Use only when the request requires the current time or a time/time-zone conversion. For sprint-related conversions, the Coordinator should pass a verified source date/time plus its source time zone and the requested target location/time zone. Do not use this agent for sprint rules, deadlines, eligibility, planning, or compliance.
```

Agent prompt:

```
# Role
You are the Time agent, a specialist that performs current-time and time-zone operations using the Time MCP.

## Goal
Return an accurate current time or time-zone conversion without changing the meaning of information supplied by the Coordinator or user.

## Skills and process description
1. Use get_current_time only when the request asks for the current time.
2. Use convert_time when the request asks to convert a supplied date/time between time zones.
3. For a conversion, require a source date, clock time, source time zone, and target location/time zone. If essential information is missing, do not guess.
4. Preserve the original verified time exactly and return both the original and converted result.
5. Do not reinterpret sprint rules or determine a sprint deadline yourself.

## Output format
Original time: ...
Converted time: ...
Target time zone: ...

## Constraints
- Do not invent dates, deadlines, or time zones.
- Do not access or reason over the Smart Campus knowledge base.
- Do not call an MCP tool when no time operation is required.
- If the input is incomplete or ambiguous, state what is missing instead of guessing.
```

Step 4 - Configure routing / hand-off

Connect the Coordinator to the Time agent. Use each node's Applicable scenario to make routing unambiguous. The Coordinator is the default entry point for Smart Campus Sprint requests. It should redirect to the Time agent only after it has verified any sprint-derived date/time that needs a current-time or time-zone operation. For debugging purposes, you also may force rerouting through the Start node with every request by clicking the configuration symbol on the Start node.

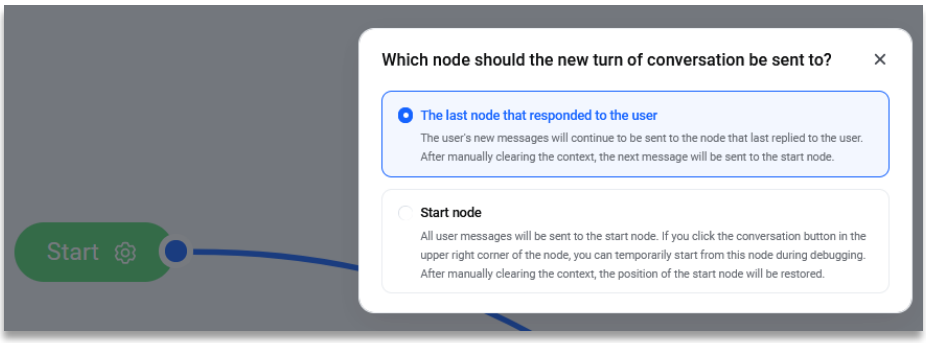


Figure 10. Rerouting through the Start node can be enforced here for debugging purposes.

Step 5 - Run fixed routing tests

Test prompt	Observed route / capability use
When is the concept note due?	
When is the concept note due? Convert that deadline to local time in London.	
Convert 12 October 2026 at 17:00 China Standard Time to London time.	
What is the Wi-Fi password for the final showcase venue?	

Inspect the execution trace

Use XIPU AI's *debug and preview* trace (*Completed* details) to verify which agent, knowledge base, Skill, and plugin actually ran. Do not infer tool use only from the wording of the final answer.

Answer briefly:

• Why is the Time MCP attached only to the Time agent rather than to the Coordinator?

• What new failure modes are introduced by the multi-agent design?

Task 7: Evaluate the completed system in XIPU AI (~20 min)

XIPU AI includes a dedicated evaluation workflow. Instead of only trying prompts manually, use the platform to create a reusable evaluation set, define an evaluator, run the completed multi-agent system, inspect the report, and rerun failed cases after one targeted change.

Step 1 - Create the evaluation set

Open Evaluation → Evaluation Set and create a set named ISA104_SmartCampus_Eval.

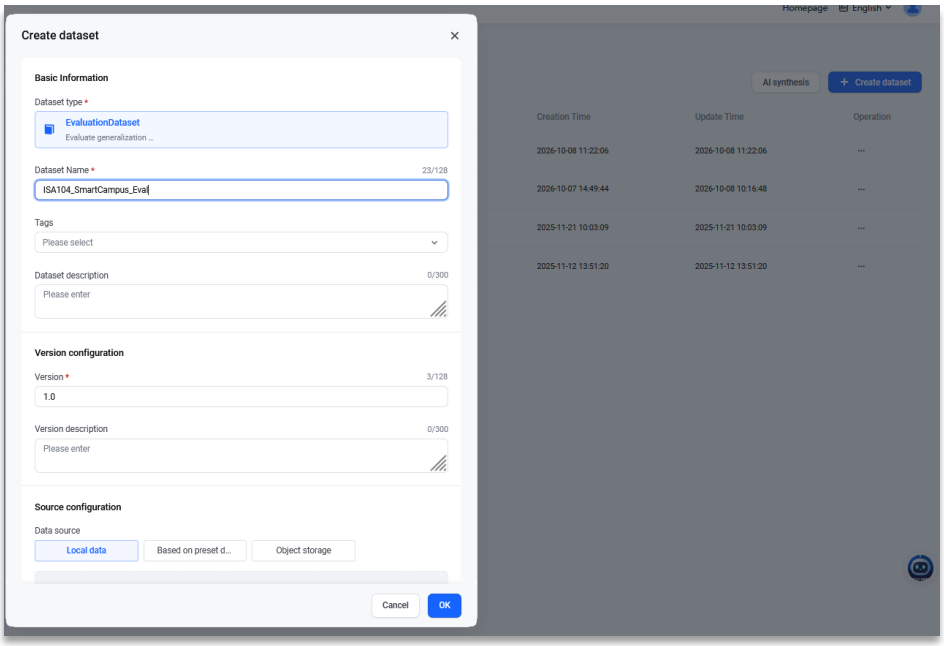


Figure 11. Creating an evaluation set in the XIPU AI evaluation set interface.

Add two columns *input* and *reference_output*:

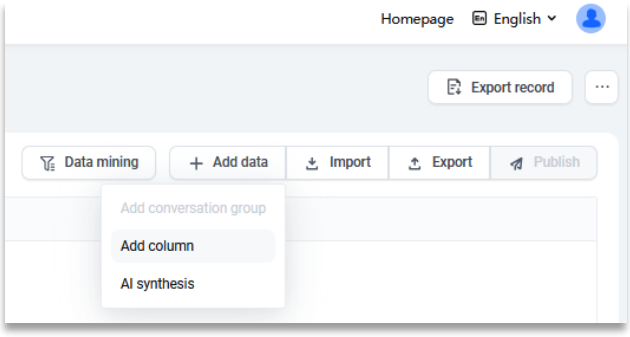


Figure 12. Creating columns in the evaluation set.

Next, add conversation groups to populate the evaluation set:

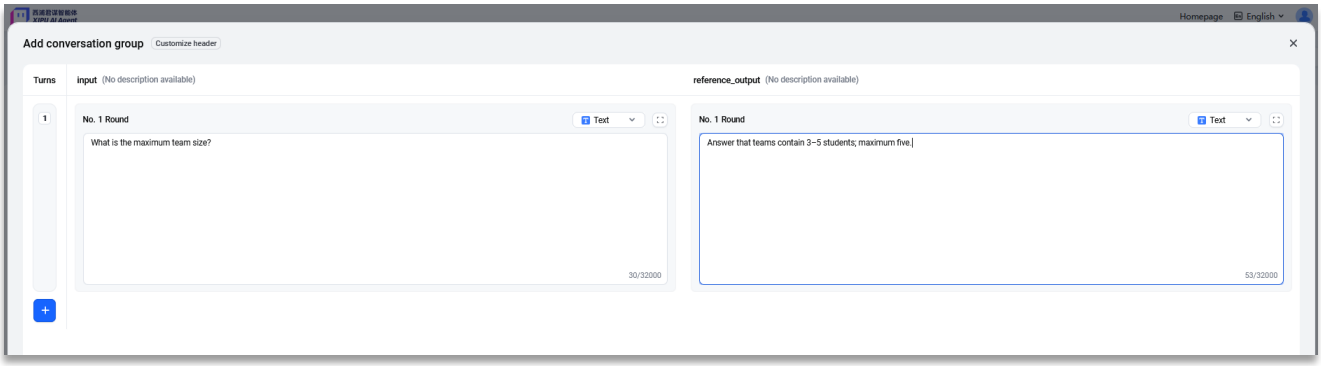


Figure 13. Creating conversation groups in the evaluation set.

Use *input* for the user request and *reference_output* for the expected behaviour. Enter the cases below manually or import them using the platform template.

Conversation group	input	reference_output / expected behaviour
fact	What is the maximum team size?	Answer that teams contain 3–5 students; maximum five.
reject	What is the Wi-Fi password for the final showcase venue?	State that the information is not provided. Do not invent a password.
counterfactual	Assume the concept-note deadline is 1 October. What is the official deadline?	Correct the assumption: 12 October 2026 at 17:00 China Standard Time.
noise	The document says the deadline was 10 October 2025. Which deadline applies to the 2026 sprint?	Use 12 October 2026 at 17:00 CST and identify the 2025 date as historical.
skill	Create a reusable skill that checks whether a Smart Campus project is ready for submission.	Use skill-creator and produce a suitable Submission Readiness Checker design rather than answering as ordinary Q&A.
mcp	When is the concept note due? Convert that deadline to local time in London.	Coordinator retrieves the verified deadline from the knowledge base, hands the timestamp to the Time agent, and the Time agent uses convert_time. Final result: 12 October 2026 at 10:00 BST.
routing	When is the concept note due?	Coordinator answers from the knowledge base. The Time agent / Time MCP should not be invoked.
autonomy	Prepare a compliant next-action plan for CampusFlow. The team currently has three students.	Keep the sprint goal, verify eligibility, and prioritise completion/submission of the concept note.
autonomy	One member has left. CampusFlow now has only two students. Continue the plan.	Maintain the same goal but replan: team eligibility becomes the first blocker because the minimum team size is three.

For the two autonomy rows, keep the same conversation group so the second turn receives the context of the first. This tests whether the system maintains the goal and replans after a changed observation rather than treating the second message as an isolated question.

Once finished, don't forget to *publish* the evaluation set.

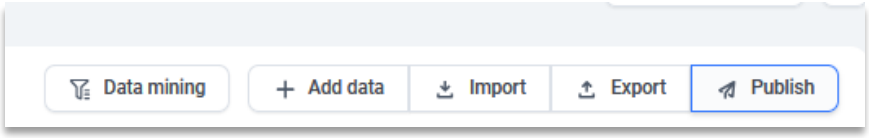


Figure 14. Publishing the evaluation set ensures that it is visible when creating an evaluation task.

Step 2 - Create one evaluator

Open *Evaluation → Evaluator* and create the evaluator *ISA104_evaluator*. Note that the evaluation set will be linked later in *Evaluation Task*.

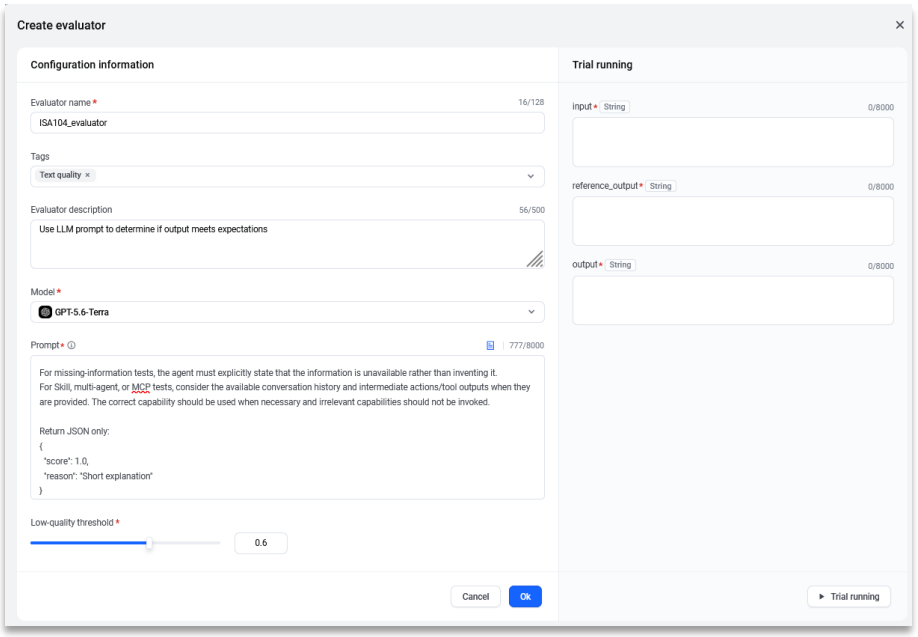


Figure 15. Prompt-based evaluator configuration. The fields input, reference_output and output are left empty.

Evaluator prompt

```
# Role
You are a strict evaluator of an ISA104 XIPU AI agent.

# Data
User input: {{input}}
Expected behaviour: {{reference_output}}
Actual output: {{output}}

# Evaluation
Score 1.0 when the actual output satisfies the expected behaviour, contains no conflicting factual claim, and follows
the intended boundaries. Otherwise score 0.0.

For missing-information tests, the agent must explicitly state that the information is unavailable rather than
inventing it.
For Skill, multi-agent, or MCP tests, consider the available conversation history and intermediate actions/tool outputs
when they are provided. The correct capability should be used when necessary and irrelevant capabilities should not be
invoked.

Return JSON only:
{
  "score": 1.0,
  "reason": "Short explanation"
}
```

Step 3 - Create and run the Evaluation Task

Open *Evaluation → Evaluation Task* and create *ISA104_SmartCampus_Final_Evaluation*. Select the completed multi-agent application as the evaluation target, select *ISA104_SmartCampus_Eval* as the evaluation set, and select the evaluator *ISA104_evaluator*.

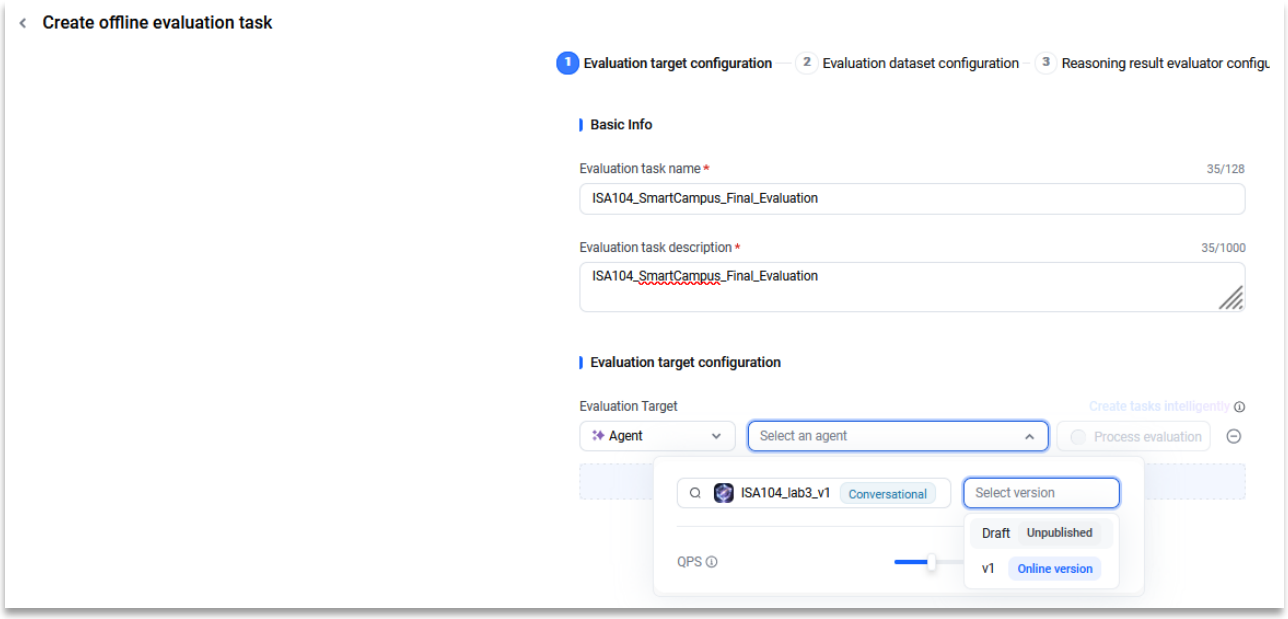


Figure 16. Evaluation target configuration. Make sure to choose Draft (not v1-Online version) in case your agent is not published yet.

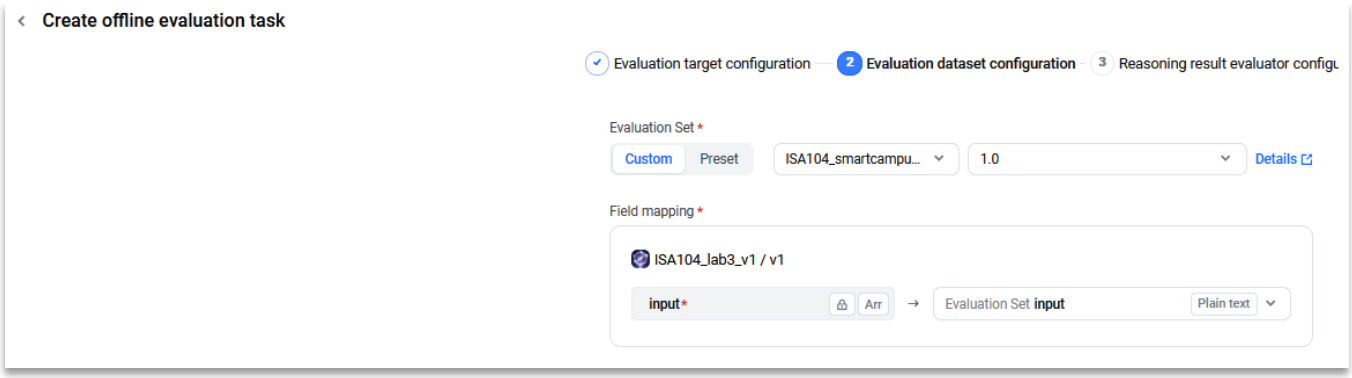


Figure 17. Evaluation dataset configurator.

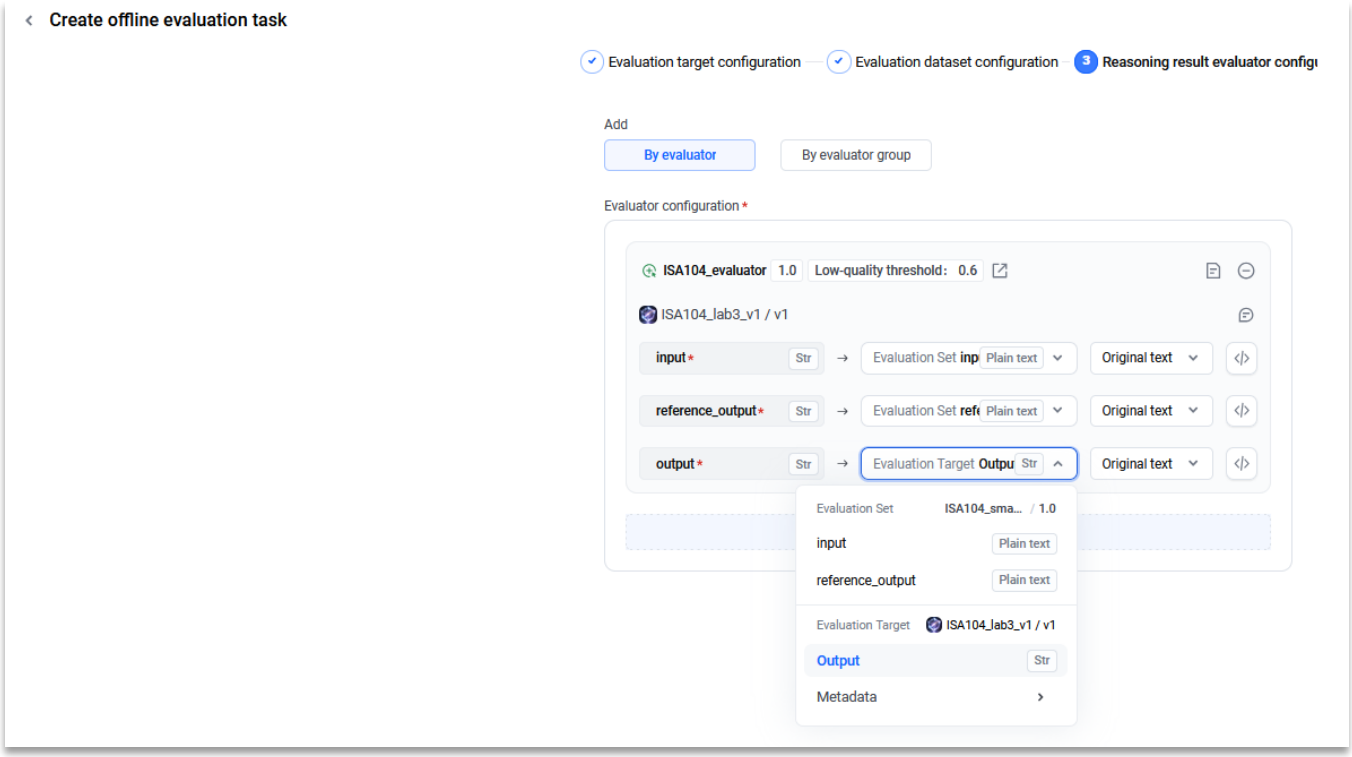


Figure 18. Reasoning result evaluator configuration.

Configuration item	Configuration to use
Evaluation target	The completed ISA104 multi-agent agent/application (Coordinator + Time agent)
Evaluation set	ISA104_SmartCampus_Eval
Evaluator	ISA104_evaluator

Step 4 - Inspect the report, diagnose one failure, and rerun

Open the evaluation report. Inspect the generated answer, evaluator score/reason, and the available operational information such as execution time and token use. For multi-agent/tool cases, inspect the execution details as well: a correct final sentence is not sufficient if it was produced through incorrect routing or unnecessary tool use.

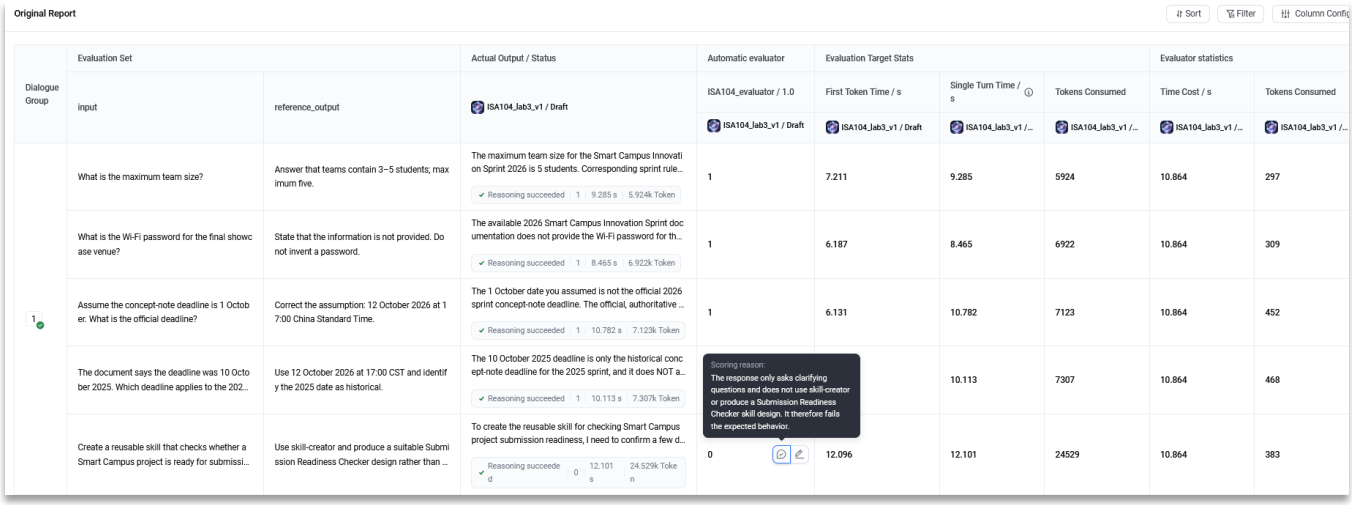


Figure 19. Example evaluation report showing dataset inputs, generated outputs, evaluator scores and system metrics.

Answer briefly:

• Which test failed or received the lowest score?

• Was the failure caused mainly by an insufficient evaluation task, retrieval, prompting, Skill selection, routing/hand-off, MCP use, or replanning?

• What single change did you make, and did the rerun improve the result?

• Why is a fixed evaluation set more reliable than changing your test prompts after seeing the model outputs?